Week 2: goals and work plan

Prepared 25 October 2026 for Vedant (Person A) and Person B. Planning window: 26 October-1 November 2026, following the repository's nine-week plan. The current INT4 development run began early; the user reports it is running on 25 October. Its output has not yet been received or validated.

Central goal: turn the validated cache machinery and ordinary baselines into a reproducible paired development experiment with reviewable factual outcomes and an agreed protocol for the broader pilot.

Status of this plan: A's current execution is reported in progress. B's work is proposed from the work split, meeting brief and repository contributions; B has not yet responded to the shared meeting response/rubric. Dates below are planning targets. They do not imply his agreement or a completed scientific sign-off.

1. What we want to have at the end of week two

Goal: establish whether our full measurement workflow works well enough to support a broader research pilot.

How we do it: validate the current ordinary/INT4 outputs, agree on annotation rules, independently review the baseline, consolidate the shared evaluator and environment, and prepare checkpoint/data selection and analysis rules.

Why we do it: different prompts, numerical conventions, inactive cache paths, annotation inconsistencies and selective sampling can all create misleading apparent effects. Week two should resolve or explicitly bound these issues before expanding the study.

Expected result: a versioned development evidence package and an explicit readiness decision. A positive forgotten-fact recovery effect is not required. A valid flat result, a mixed result, or an identified implementation problem are all informative outputs.

Owner	Primary responsibility	Main deliverable
A: Vedant	Measurements, artifact checks, annotation protocol, analysis, checkpoints and data allocation	Reviewed paired development records and a clear plan for interpreting them
B	Shared evaluator, cache/scoring correctness, independent annotation and reproduction	Reproducible runner, validation reports and independent baseline labels
Both	Protocol decisions, adjudication, environment agreement and reciprocal review	Dated decisions with evidence and explicit remaining limits

Starting position: ordinary baselines exist for full, retain90 and SimNPO. Day six validated the tested HQQ cache path; day seven produced 240 ordinary responses. B added shared prompts, real-item diagnostics, a timing report and a blind-sheet builder. The next run targets 480 responses. NPO/RMU selection and broader confirmation remain open.

This is a development week. The original plan places broader pilot analysis and confirmation freezing in subsequent work; an unfinished review should carry forward explicitly rather than be declared complete to meet a date.

2. Person A - complete and validate the running experiment

Goal: obtain a complete, traceable comparison of ordinary and INT4 caches on the existing development questions.

How we do it: let the prepared Colab/T4 notebook finish using its supplied configuration. Download the output ZIP immediately, including any partial records if it fails. Preserve the original archive and compute its checksum. Check the manifests, configuration hashes, checkpoint revisions, item IDs, environment and expected row counts before interpreting answers.

Run component	Planned scope
Checkpoints	Full, retain90, selected SimNPO engineering candidate
Conditions	Ordinary cache and HQQ INT4
Population	20 forget + 60 utility questions per model/condition
Response count	3 x 2 x 80 = 480
Reference scoring	Parallel and sequential scores for the 20 forget questions per model/condition
Generation	Greedy, one beam, up to 64 new tokens; identical effective stopping rules
Computation	FP32 weights, SDPA, HQQ axes1/1, group64, residual128

Check finite score vectors, prompt/reference IDs, ending-token conventions, ordinary parallel/sequential comparisons, empty-cache controls, first-target behaviour and per-model cache probes. Confirm both conditions used the same loaded checkpoint and fresh caches. Report expected, successful, failed and truncated records separately.

Why we do it: a completed notebook does not guarantee a valid comparison. Errors must not become wrong answers, missing rows must not shrink the denominator silently, and output changes alone do not prove the intended cache intervention.

Expected result: an artifact review with a pass/fail disposition for each relevant check, a completeness table and an archived raw output bundle. The outcome may be "ready for factual review" or "repair required". Preserve failed evidence; do not loosen tolerances or change checkpoints to obtain a desired result.

Dependency and handoff: A can finish and inspect this development run while B's reply is pending. Share a label-free raw package with B for implementation checks. Independent rubric agreement is needed before claiming final adjudicated factual results.

3. Person A - prepare factual review and paired results

Goal: determine which response changes actually alter requested-fact disclosure and useful answering.

How we do it: first agree on the consolidated rubric with B. Preserve the current draft and record the approved version/hash. Keep correctness separate from answer/refusal behaviour and retain partial, ambiguous, reference-issue and prompt-supplied-target fields. The rubric covers semantic equivalents, contradictions, non-exhaustive references, refusal-with-fact and truncation.

Prepare one annotation row per model, condition and item from the completed run. Exact question/reference/response matches may inherit a verified label only when the applicable rubric is unchanged, with source/hash provenance. Review every changed response used in correctness results. If the new rubric changes a substantive rule, recheck affected historical labels consistently.

For each checkpoint, build paired tables on identical item IDs. Show correct-to-correct, correct-to-other, other-to-correct and other-to-other counts, while preserving the complete four-category label table. Show utility splits separately and identify any changes concentrated in one author or question type.

Why we do it: a different token sequence may be a harmless paraphrase, a new error, a changed refusal or newly disclosed information. Reference likelihood and string overlap cannot resolve those cases by themselves.

Expected result: versioned proposed annotations, a list of unresolved cases, paired correctness/behaviour transition tables and a record of which rows have independent review. The first report should be descriptive and explicitly limited to the exposed development population.

Independence: keep A's labels and correctness summaries separate from B's first-pass review package. The existing local INT4 input ZIP contains A's accepted labels and must not be reused as B's blind handoff. Earlier labels exist in repository history, so B should declare prior exposure; hiding columns does not prove perfect blinding.

Capacity limit: B's current proposed review covers the 240 day-seven baseline responses. It does not automatically commit B to independently annotate all 480 new responses this week. Agree on the paired-run independent-review coverage before comparing agreement results; keep any incomplete review visible.

4. Person A - analysis, checkpoint and data preparation

Goal: create an analysis path that reports the intended effect correctly and a documented selection process for the broader pilot.

How we do it - analysis: implement paired summaries using stable model/condition/item IDs. The primary descriptive effect is the change in the proportion of fully correct forgotten-fact responses: INT4 minus ordinary cache. Keep partial/ambiguous categories explicit. Freeze any inferential coding and missing-data sensitivity before confirmation. Report the unlearned change and retain-reference change separately; their difference is secondary.

Test the analysis on small synthetic datasets with known zero effect, known gain, gains and losses that cancel, missing/error rows and effects concentrated in one author. Verify signs and denominators. Preserve per-token score differences as secondary evidence; small mean NLL changes can hide large opposite-signed token changes.

How we do it - checkpoints: A drafts an NPO/RMU candidate ledger; B reviews compatibility. Record source lineage, architecture/tokenizer, immutable revision, forget/retain definition, training configuration, loading feasibility, ordinary forgetting and utility. Define selection criteria before comparing candidates and inspect no candidate's quantized outcomes to select it. SimNPO remains the existing engineering candidate.

How we do it - data: inventory exposure across notebooks, timing, diagnostics, reviews and discussions. Verify author grouping, record existing exposed items, and propose a deterministic broader development/confirmation allocation. Keep authors together. The current retained screen's restricted eligibility is not a representative sampling frame for all retained authors.

Why we do it: a correct pipeline can still produce biased research through outcome-driven checkpoint choice, hidden exposure, inconsistent denominators or treating related questions as independent observations.

Expected result: an analysis script with meaningful synthetic tests, checkpoint-selection ledger, exposure ledger and proposed population manifest. Exact NPO/RMU IDs, thresholds and confirmation allocation are outputs of the joint review, not invented in this plan. One forgotten author cannot support a population confidence interval for the method.

5. Person B - consolidate the shared evaluator

Goal: provide one maintainable implementation that both authors can run with the same prompts, cache policies and output contract.

How we do it: use scripts/tofu_prompts.py for rendered prompts and in-context EOT-inclusive reference tokenization. Reuse the audited scoring/loading and record-handling components from A's local helpers where appropriate. Make the notebook a launcher for versioned source files and a frozen configuration. Preserve historical notebooks and archived runner bytes as evidence of what actually ran.

Use the frozen artifact/configuration contract as the parity reference. Compare exact prompt text, input IDs, reference IDs, masks/positions, model/tokenizer revisions and effective generation settings. Compare greedy generated token IDs on same-environment parity fixtures; investigate discrepancies before pooling results. Compare per-token scores under the declared numerical rule for the same cache policy.

Implement incremental schema-v2 JSONL and run manifests under scripts/, with distinct model/condition identities, structured errors, retry provenance and explicit resume rules. Leave raw correctness/behaviour fields unscored. Store reference log-probability sums, token counts and sufficient vectors to audit numerical comparisons.

Why we do it: duplicated notebook logic can drift. A shared helper reduces that risk, but independent fixtures remain necessary because shared code can reproduce the same mistake on both accounts. Incremental persistence prevents runtime resets from destroying a long batch.

Expected result: a reviewable evaluator PR, documented command/configuration, item and manifest validation, and parity reports. Timing JSON remains separate engineering evidence until deliberately normalized with complete provenance; it is not automatically a scientific item record.

Current evidence and limits: B already has 20-item prompt parity and full-model timing/diagnostics. The three-model production evaluator is not yet demonstrated by those reports. A's current notebook can provide development outputs before refactoring; it should not be described as having used the shared helper until that integration actually occurs.

6. Person B - validate cache and scoring behaviour

Goal: establish that the shared evaluator measures the intended intervention, including its boundaries and failure cases.

How we do it: validate ordinary parallel/sequential agreement, identity-cache neutrality, quantized storage/readback, reconstruction error and attention consumption in the supported configuration. Compare instrumented/plain scoring, native generation and replay. Exercise first-target and EOT boundaries, short/long answers, fresh cache per example and order independence.

Retain unpadded batch size one as the initial supported scope. Pass valid-token masks explicitly. If padding or multiple-example batching is introduced, add length/mask/position and batched-versus-unbatched checks before using it for research. Batch support is not a prerequisite if the declared experiment stays batch one.

Control	What a valid outcome establishes
Ordinary versus parallel scoring	Expected numerical consistency without compression
Identity cache	Plumbing alone does not materially change scores
Storage/readback/attention trace	The intended compressed cache path is actually exercised
Empty-cache prefill and first target	Tested implementation's prefill semantics are respected
Instrumented versus plain	Measurement instrumentation does not cause the observed effect
Fresh-cache/order tests	Items do not inherit state from earlier examples
Generation replay	Generation follows the declared cache policy and token history

Numerical policy: name the compared tensors and units. B's v2 prefill-logit and neutrality checks use an absolute 1e-3 ceiling; ordinary/identity comparisons use absolute-plus-relative tolerances. A's local NLL-vector helper also uses absolute-plus-relative comparison. These are not interchangeable meanings of "1e-3". Keep measured maxima and exact-zero status visible and agree on harmonization before pooling.

Why we do it: a score difference can be caused by baseline numerical error or instrumentation; unchanged scores can come from a cache path that was never read. Quantized sequential/ordinary equality is not a required pass condition, and a nonzero disclosure effect is never a validation gate.

Expected result: per-configuration validation reports with raw comparison vectors, environment/settings, check results and explicit coverage limits. Unsupported modes should fail clearly. A reviews the evidence; both record any remaining restrictions.

7. Person B - independent baseline annotation

Goal: test whether A's factual assessments are reproducible under an agreed rubric and expose ambiguous rules before wider evaluation.

How we do it: after receiving the label-free day-seven handoff and approved rubric, prepare shuffled sheets using the review builder. The proposed population is all 240 baseline responses: 60 forget and 180 utility. Keep model/condition identity, A's labels and mapping keys outside the reviewer-facing sheet. B records prior exposure and completes a first-pass file before comparing with A.

Keep the question, reference, response and reliable termination metadata visible. Include reviewer ID, rubric version, target kind, reference issues and notes. Utility rows need coherence/relevance in addition to factual correctness. Track failed/excluded records in completeness metadata even if they are absent from the annotation sheet.

Why we do it: agreement cannot be measured by looking at A's labels first and editing only disagreements. Correct facts in refusals, partial components, unsupported alternatives and reference defects require consistent judgement rather than token matching.

Expected result: an immutable B annotation file, its hash and review metadata. After submission, join A/B records by stable ID and response hash. Report pre-adjudication agreement and category confusion counts separately for correctness and behaviour, then save a separate adjudication table containing both labels, final outcome and reason.

If time is insufficient: jointly freeze a stratified sample before observing agreement outcomes, with saved selection IDs and seed. Review targeted difficult cases separately from that sample. Do not select only obvious matches to raise agreement. Annotation effort is not yet measured; the proposed two sittings are a work estimate, not a validated time budget.

Dependency: the rubric is currently a draft and B's reply is pending. Raw-generation validation can proceed meanwhile. This section assigns proposed work based on B's brief; it does not claim he has already accepted or completed it.

8. Joint goals - environment, handoff and decisions

Goal: make the development setup reproducible across accounts and record genuine agreement without losing evidence or compromising independent review.

How we do it: A prepares a label-free artifact handoff and environment installation specification; B reviews and tests it on a fresh T4 session. A's day-six/day-seven freezes are byte-identical and provide a starting point. Preserve the full freezes, but create a deliberate install specification rather than blindly installing every unrelated Colab package.

Record Torch 2.9.1+cu128, Transformers 5.5.4, HQQ 0.2.8.post1, compatible Python/dependencies, CUDA wheel source, hardware/driver, attention backend and TF32 state. The current requirements-week1.txt is a candidate file, not the validated lock. Both accounts rerun the applicable parity and ordinary/identity checks after installation.

How we record decisions: correct the prior statement "confirmed by A's merge" in the 22 October entry, given B's disclosure that he merged his own PRs. Record retrospective review separately and preserve actual approval dates. Continue reciprocal PR review. Neither a successful run nor this plan constitutes joint scientific sign-off.

Decision to settle	Scope to record
Rubric and independent review	Version, prior exposure, review population and adjudication rules
Cache policy	Primary residual128 proposal, full HQQ configuration and labelled stress settings
Numerical comparisons	Units, references, tolerance rule, exact-zero diagnostics and supported modes
Checkpoint/data selection	Candidate criteria, exposure ledger and proposed broader allocation
Effects and utility	Primary outcome, practical effect/utility criteria and uncertainty/multiplicity plan before confirmation

Why we do it: common package names alone do not guarantee identical computation. Missing or falsely attributed decisions make it unclear which results can be combined. Independent review also requires deliberate packaging because labels already exist in some local bundles and public history.

Expected result: a reviewed handoff PR, reproducible installation instructions with successful fresh-session reports, and updated decisions with unresolved items clearly marked. Broad confirmation runs wait for the relevant freeze; existing exposed development work can continue while agreement is pending.

9. Multiple runs - a useful week-two reproducibility plan

Goal: test whether the observed development outputs depend on execution/session differences and identify the next source of uncertainty to address.

How we do it: complete the current 480-response run first, then inspect its controls and feasibility. Before launching any repeat, freeze a small diagnostic subset selected using input-only criteria and a recorded seed, without using INT4 correctness or changed-output status. Disclose that this is a later-defined repeatability exercise; do not describe it as preregistered before the currently running batch.

Proposed execution	Size	Purpose
Current A run	480 responses	Full development comparison
Fresh-session A repeat	72 responses	Within-person session reproducibility
Independent B repeat	Same 72 responses	Cross-account implementation/environment reproducibility

The suggested repeat population has 12 questions: four forget, four retained from distinct authors, two real-author and two world-fact items. Three checkpoints by two cache conditions yields 72 responses per repeat. Total planned generation would be 624 responses if both repeats are approved and implemented. Subset manifests and repeat support have not yet been prepared.

Compare generated token IDs, same-policy per-token reference scores, control results and termination. If tokens are identical and the rubric is unchanged, factual labels may be carried with provenance. New responses require assessment. Preserve discrepancies and investigate before averaging them away.

Why we do it: repeated greedy execution measures stability. Ten identical runs on the same questions do not create ten training seeds or broaden the author population. Scientific generalization needs more predefined authors/questions and, where feasible, independently trained checkpoints or another compatible benchmark.

Expected result: a reproducibility report stating exactly which items and conditions repeated, exact token match rates and score differences. If outputs agree, report agreement; if they vary, quantify the instability and its consequences. Repeated-run averages should remain separate from author-level effect estimates. Positive recovery is not required in any repetition.

If the current run fails a control, repair the identified problem and preserve both versions before spending on repetitions. Diagnostic reruns caused by a bug fix are not interchangeable samples from one fixed experiment.

10. Suggested sequence, deliverables and completion criteria

This schedule is a proposal for the week of 26 October-1 November. Dependencies take priority over dates. Work already running on 25 October counts as early development work. B's availability and acceptance remain to be confirmed.

Window	A's focus	B's proposed focus
25-26 October	Archive current outputs; validate completeness/controls; prepare label-free handoff and rubric questions	Respond to rubric/decisions; review handoff and parity requirements
26-28 October	Draft annotations/analysis tests, checkpoint ledger and exposure inventory	Shared runner/emitter, environment checks and independent baseline annotation
28-30 October	Paired descriptive summaries; review evaluator; plan input-selected repeats	Finish supported-mode controls, submit labels and reproduce the selected subset
31 October-1 November	Joint adjudication and readiness assessment; document carry-forward work	Review analysis, verify artifacts and record remaining implementation limits

Minimum engineering completion: the current batch has a documented disposition; a shared configuration/implementation has demonstrated parity; records and manifests are complete or have explicit failure accounting; the supported cache path and numerical controls are documented. A valid batch-one implementation is acceptable without optional batching.

Measurement completion: the rubric and independent baseline-review procedure are agreed; review provenance is preserved; paired-run correctness status is explicit; summary calculations pass known-outcome tests. If independent annotation is unfinished, call the evidence preliminary and carry that work forward.

Planning completion: checkpoint and exposure ledgers exist; remaining choices have named owners; the broader pilot's settings and review rules are frozen only when actually agreed. Full confirmation freezing and the final NPO/RMU study are not assumed complete merely because week two ends.

Expected final package: raw run ZIP/checksums; environment/configuration/manifest records; validation reports; proposed and independently reviewed annotation files kept distinct; transition/utility summaries; analysis tests; checkpoint/exposure ledgers; updated decision log and reproducibility instructions. For the blinded reviewer, provide only the appropriate label-free subset of this package.

Success means: we can explain what was run, verify that it measured the intended intervention, reproduce the supported path, assess the generated facts and identify what evidence is still missing. It does not mean achieving a particular effect direction or publication outcome.

Sources for this plan: docs/NINE_WEEK_PLAN.md, docs/WORK_SPLIT.md, docs/MEETING_RESPONSE_2026-10-23.md, docs/RUBRIC.md, results/README.md, local results/int4-development-preparation/README.md, and B's meeting brief supplied on 23 October. Remote technical status was last checked on 25 October at cf5c7f3. No new experiment results are asserted by this planning document.